

What is a PRIMARY KEY?

A PRIMARY KEY (PK) is a constraint that uniquely identifies every row in a table.

Why is it called "Primary"?

A table may have several candidate keys:

- StudentID
- Email
- AadhaarNumber

All may uniquely identify a student. We choose one as the main identifier.

A Primary Key has three important properties.

1. UNIQUE

No duplicates. Allowed

2. NOT NULL

Every row must have a value.

3. Only One PRIMARY KEY Per Table

Syntax

Inline

```
CREATE TABLE Students(  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(100)  
);
```

Table-Level

```
CREATE TABLE Students(  
    StudentID INT,  
    Name VARCHAR(100),  
  
    PRIMARY KEY(StudentID)  
);
```

Composite Primary Key

```
CREATE TABLE Enrollment(  
    StudentID INT,  
    CourseID INT,  
  
    PRIMARY KEY(StudentID, CourseID)  
);
```

Internal Working

Automatic Unique Index

When a Primary Key is created, MySQL automatically creates a unique index.

- Without an index, Database scans every row.
- With a Primary Key index, Database directly locates the row.

This makes lookups extremely fast.

Clustered vs Non-Clustered

Interview favorite

A clustered index dictates the physical sorting of the data itself on disk and acts as the table itself, meaning you can only have one per table.

A non-clustered index acts like a book's back-of-the-book index, creating a separate data structure that contains pointers back to the physical data rows.

Physical Storage & Quantity per Table:

You can have only one clustered index per table (since data can only be physically sorted one way), but you can have up to 999 non-clustered indexes.

Clustered indexes are significantly faster for range-based queries (e.g., retrieving orders between dates).

Non-clustered indexes are typically faster for exact match lookups (e.g., searching for a specific email address).

MySQL (InnoDB)

The Primary Key is a clustered index.

SQL Server

Also supports clustered and non-clustered indexes. By default, the Primary Key is usually created as a clustered index unless specified otherwise.

Changing Primary Keys

Example

Drop old PK

```
ALTER TABLE Students  
DROP PRIMARY KEY;
```

Create new PK

```
ALTER TABLE Students  
ADD PRIMARY KEY(Email);
```

Usually avoid changing primary keys in production because they are often referenced by foreign keys. If foreign keys reference this primary key, the operation will fail until those foreign keys are handled.

FOREIGN KEY

5.1 Introduction

Referential Integrity

A FOREIGN KEY ensures that relationships between tables remain valid.

Suppose

Customers

CustomerID
1
2

Orders

OrderID	CustomerID
101	1
102	2

Every order must belong to an existing customer.

Parent-Child Relationship

- **Parent:** Customers
- **Child:** Orders

Orders references Customers.

Why Foreign Keys Exist

Without a Foreign Key

Orders

OrderID	CustomerID
101	999

Customer 999 doesn't exist.

Bad data.

Foreign Keys prevent this.

Syntax

Inline

```
CREATE TABLE Orders(  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    FOREIGN KEY(CustomerID)  
        REFERENCES Customers(CustomerID)  
);
```

Table-Level

```
CREATE TABLE Orders(  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
  
    CONSTRAINT fk_customer  
    FOREIGN KEY(CustomerID)  
    REFERENCES Customers(CustomerID)  
);
```

Named constraints make maintenance easier.

INSERT Rules

Parent first.

```
INSERT INTO Customers  
VALUES(1, 'Aadya');
```

Then

```
INSERT INTO Orders  
VALUES(101, 1);  
  
INSERT INTO Orders  
VALUES(101, 999);
```

Rejected because customer 999 doesn't exist.

UPDATE Rules

Suppose Orders reference CustomerID = 1

Trying to update the parent key

```
UPDATE Customers  
SET CustomerID = 5  
WHERE CustomerID = 1;
```

Behavior depends on the foreign key action (ON UPDATE). With the default restrictive behavior, this update is rejected because child rows still reference the old value.

DELETE Rules

Delete customer



Orders still exist.

Database checks the foreign key action.

If no cascading behavior is configured, the delete is rejected to avoid orphan rows

CASCADE

Delete parent



Delete children automatically.

SET NULL

Delete parent



Foreign key column becomes NULL.

Requires the foreign key column to allow NULL.

SET DEFAULT

Supported by some databases.

The child foreign key is set to its default value.

Not supported by MySQL InnoDB.

ON DELETE

Example

```
CREATE TABLE Orders(  
  OrderID INT PRIMARY KEY,  
  CustomerID INT,  
  
  FOREIGN KEY(CustomerID)  
  REFERENCES Customers(CustomerID)  
  ON DELETE CASCADE  
);
```

Delete customer



Orders automatically deleted.

ON UPDATE

ON UPDATE CASCADE

Parent ID changes



Child IDs automatically updated.

Self-Referencing Foreign Keys

Employees

EmployeeID	ManagerID
1	NULL
2	1
3	2

```
CREATE TABLE Employees(  
    EmployeeID INT PRIMARY KEY,  
    ManagerID INT,  
    FOREIGN KEY (ManagerID)  
        REFERENCES Employees(EmployeeID)  
);
```

This models organizational hierarchies.

Composite Foreign Keys

- **Parent:** (StudentID, CourseID)
- **Child:** (StudentID, CourseID)

Both columns together reference the composite primary (or unique) key.

Disabling Foreign Key Checks

MySQL

```
SET FOREIGN_KEY_CHECKS = 0;  
-- migration work  
SET FOREIGN_KEY_CHECKS = 1;
```

Why are foreign keys used?

To enforce relationships between tables and prevent inconsistent or orphaned data. FOREIGN KEY references a primary (or unique) key in another (or the same) table to create relationships.

ON DELETE CASCADE and ON DELETE SET NULL?

- CASCADE deletes child rows automatically.
- SET NULL keeps the child row but sets the foreign key column to NULL.